

Operating Systems

Unit 1 — MCQ Question Bank with Answers

Question and options on the left, answer with a one-line reason on the right. Questions grouped by topic (original numbering not kept).

OS Services, System Calls, Linkers and Loaders

Question	Answer
1. Which of the following is NOT a service provided by an operating system? a) Program execution b) Error detection c) Data warehousing d) I/O operations	c) Data warehousing OS services: program execution, I/O, file systems, communication, error detection, resource allocation, protection. Data warehousing is an application.
2. What does the user interface of an operating system primarily allow? a) Direct access to hardware components b) Communication between user and system c) Compilation of programs d) Direct execution of binary machine code	b) Communication between user and system CLI / GUI / batch interfaces let the user issue commands to the OS.
3. Which type of system call is used to terminate a process? a) fork() b) wait() c) exit() d) read()	c) exit() exit() ends the calling process; fork() creates, wait() waits for a child, read() is file I/O.
4. System services are provided by the operating system to: a) Execute programs without user intervention b) Manage database operations c) Facilitate communication between processes d) Allow direct hardware access	c) Facilitate communication between processes Communication (IPC) is one of the standard OS services; direct hardware access is exactly what the OS prevents.
5. Which of the following is a system call used for file manipulation? a) open() b) malloc() c) print() d) shutdown()	a) open() File-management calls: open, read, write, close, create, delete. malloc/print are library functions.
6. A linker is primarily used to: a) Translate source code into machine code b) Combine object code into a single executable c) Execute programs on a CPU d) Manage I/O operations	b) Combine object code into a single executable The linker resolves references between object files and libraries to produce one executable.
7. What is the primary function of a loader in an operating system? a) To load files from external drives b) To allocate memory for programs c) To load executable files into memory for execution d) To translate high-level language into machine language	c) To load executable files into memory for execution The loader copies the executable into memory, relocates addresses and starts it.
8. Which of the following is a type of user interface provided by an operating system? a) Command Line Interface (CLI) b) HTML Interface c) Script Engine Interface d) Media Player Interface	a) Command Line Interface (CLI) OS user interfaces: CLI (shell), GUI, touch/batch interfaces.
9. Which of the following system calls is used for process control in an operating system? a) shmget() b) exec() c) printf() d) malloc()	b) exec() Process-control calls: fork, exec, exit, wait, abort. shmget is IPC, printf/malloc are library calls.

Question	Answer
10. What is the main purpose of system services in an operating system? a) To enable user programs to directly interact with the hardware b) To provide a layer of abstraction between user applications and the hardware c) To compile user programs d) To format the hard drive	b) To provide a layer of abstraction between user applications and the hardware Services hide hardware details behind a uniform interface (system calls).
11. What is the primary purpose of system calls in an operating system? a) To design GUI of the operating system b) To provide an interface for user-level applications to access OS services c) To compile programs d) To encrypt data	b) To provide an interface for user-level applications to access OS services A system call is the programming interface to kernel services (trap from user mode to kernel mode).

Processes and CPU Scheduling

Question	Answer
12. Which of the following defines a process? a) A program in execution b) A passive entity like a file c) A memory management technique d) A type of hardware	a) A program in execution A program on disk is passive; a process is the active instance with its own PC, registers, stack, heap.
13. What is the primary function of a Process Control Block (PCB)? a) Manages file I/O operations b) Stores information about a process c) Controls memory allocation d) Controls CPU scheduling	b) Stores information about a process PCB holds state, PC, registers, scheduling info, memory limits, open files, accounting.
14. Which state does a process enter when it is waiting for an I/O operation to complete? a) Ready b) Waiting c) Running d) Terminated	b) Waiting Running → Waiting (blocked) on an I/O or event request; back to Ready when it completes.
15. What is context switching in process management? a) Switching from user mode to kernel mode b) Saving and restoring the state of a process c) Switching from one memory block to another d) Changing scheduling algorithms	b) Saving and restoring the state of a process The kernel saves the current process's PCB and loads the next one's; pure overhead.
16. In which state does a process use the CPU for execution? a) Ready b) Running c) Waiting d) New	b) Running Only one process per CPU core is in the Running state at a time.
17. Which scheduling algorithm selects the process that arrives first? a) First-Come, First-Served b) Shortest Job First c) Round Robin d) Priority Scheduling	a) First-Come, First-Served FCFS is a simple FIFO ready queue; non-preemptive, suffers from the convoy effect.
18. What is the goal of process scheduling in a multiprogramming environment? a) Maximizing CPU idle time b) Maximizing process throughput c) Minimizing process execution d) Minimizing memory usage	b) Maximizing process throughput Keep the CPU busy and complete as many processes per unit time as possible.
19. Which of the following is a preemptive scheduling algorithm? a) Shortest Job First b) Round Robin c) First-Come, First-Served d) Priority Scheduling (non-preemptive)	b) Round Robin RR preempts on time-quantum expiry. (SJF and priority also have preemptive versions – SRTF, preemptive priority – but as listed RR is the answer.)

Question	Answer
20. What is the turnaround time of a process? a) The total time taken by a process from submission to completion b) The time spent in the ready queue c) The time spent waiting for I/O d) The time spent executing instructions	a) The total time taken by a process from submission to completion Turnaround = completion – arrival = waiting + burst (+ I/O). (b) is waiting time.
21. In Round Robin scheduling, if the time quantum is too small, the algorithm will: a) Act like First-Come, First-Served b) Cause high context switching overhead c) Cause low CPU utilization d) Minimize response time	b) Cause high context switching overhead Too many switches waste CPU; too large a quantum degenerates into FCFS.
22. Which of the following is the main objective of CPU scheduling? a) Minimizing context switches b) Maximizing CPU utilization c) Reducing process memory usage d) Minimizing process turnaround time	b) Maximizing CPU utilization Criteria: max CPU utilization & throughput, min turnaround, waiting and response time; utilization is the primary one.
23. What is a process state transition from running to waiting called? a) I/O Burst b) Preemption c) Process Swap d) Blocking	d) Blocking The process blocks itself waiting for I/O/event; preemption is Running → Ready.
24. In a multiprocessor system, what is the benefit of load balancing in scheduling? a) Increases the number of context switches b) Ensures an even distribution of work among processors c) Lowers memory consumption d) Improves process priority	b) Ensures an even distribution of work among processors Push/pull migration keeps all CPUs busy so no processor idles while others are overloaded.
25. In which state does a process wait for an event to occur? a) Ready b) Running c) Waiting d) New	c) Waiting Waiting (blocked) state; Ready means waiting only for the CPU.
26. Which scheduling algorithm assigns a fixed time slice to each process? a) First-Come, First-Served b) Shortest Job First c) Round Robin d) Priority Scheduling	c) Round Robin Each process gets one time quantum (typically 10–100 ms) in circular order.

Inter-Process Communication and Synchronization

Question	Answer
27. What is inter-process communication (IPC)? a) The mechanism by which processes exchange data b) The mechanism by which processes share the same PCB c) The technique to switch between processes d) A hardware interrupt signaling technique	a) The mechanism by which processes exchange data Two models: shared memory and message passing.
28. Which of the following is an example of inter-process communication using shared memory? a) Using semaphores b) Using a message queue c) Using pipes d) Processes sharing a common memory segment	d) Processes sharing a common memory segment A shared segment (e.g. shmget/shmat) is mapped into both address spaces; queues and pipes are message passing.
29. What is the purpose of synchronization in inter-process communication? a) To allocate CPU time to each process b) To avoid race conditions when processes share resources c) To stop processes from terminating d) To allocate memory efficiently	b) To avoid race conditions when processes share resources Without coordination the result depends on the interleaving of accesses (race condition).

Question	Answer
30. Which of the following IPC methods allows two processes to send messages to each other? a) Shared Memory b) Message Passing c) Paging d) Swapping	b) Message Passing send()/receive() primitives; useful across machines as well.
31. In message-passing IPC, what is the key characteristic? a) No shared memory between processes b) Processes must share memory c) It only works for I/O-bound processes d) Messages cannot be stored in queues	a) No shared memory between processes Data is copied through the kernel; no shared address space is needed.
32. What is a critical section in inter-process communication? a) A section of code where processes access shared resources b) The main program code executed by the OS c) A memory segment allocated to one process d) A function that executes I/O operations	a) A section of code where processes access shared resources Only one process may be in its critical section at a time (mutual exclusion).
33. Which of the following is the purpose of a semaphore in IPC? a) To signal process termination b) To synchronize access to shared resources c) To allocate memory to processes d) To increase CPU utilization	b) To synchronize access to shared resources Integer variable accessed only through wait()/signal() (P/V) for mutual exclusion and ordering.

Threads and Multithreading Models

Question	Answer
34. Which multithreading model maps many user threads to one kernel thread? a) Many-to-One b) One-to-One c) Many-to-Many d) Two-Level Model	a) Many-to-One All user threads are multiplexed on a single kernel thread (e.g. Green threads).
35. In the One-to-One multithreading model, how are user threads mapped to kernel threads? a) One user thread maps to many kernel threads b) One user thread maps to one kernel thread c) Many user threads map to one kernel thread d) Many user threads map to many kernel threads	b) One user thread maps to one kernel thread Every user thread gets its own kernel thread (Linux, Windows).
36. What is a disadvantage of the Many-to-One multithreading model? a) It uses too many kernel threads b) A blocking call in one thread can block the entire process c) It increases the overhead of context switching d) It cannot support concurrency	b) A blocking call in one thread can block the entire process Only one kernel thread exists, so one blocking system call stalls all user threads; also no true parallelism on multicore.
37. Which multithreading model allows multiple user threads to be mapped to multiple kernel threads? a) Many-to-One b) One-to-One c) Many-to-Many d) Single-to-Many	c) Many-to-Many Multiplexes m user threads onto $n \leq m$ kernel threads; combines the advantages of both.
38. In the Two-Level Model, what can user-level threads do that kernel-level threads cannot? a) Make system calls b) Be created faster and with less overhead c) Execute in kernel mode d) Access hardware directly	b) Be created faster and with less overhead User threads are managed by a library without kernel involvement, so creation/switching is cheap.
39. In which multithreading model does the operating system create a separate kernel thread for each user thread? a) Many-to-One b) One-to-One c) Many-to-Many d) Two-Level	b) One-to-One Provides more concurrency, at the cost of a kernel thread per user thread.

Question	Answer
40. What is a disadvantage of user-level threads? a) They are slower to create than kernel-level threads b) They require system calls for thread management c) They cannot leverage multiprocessor systems efficiently d) They can only be scheduled preemptively	c) They cannot leverage multiprocessor systems efficiently The kernel sees a single thread, so it can run on only one processor at a time.
41. What happens when a thread makes a blocking system call in a Many-to-One multithreading model? a) Only the calling thread is blocked b) The entire process is blocked c) The OS creates a new thread to handle it d) The thread continues execution	b) The entire process is blocked The single kernel thread blocks, so every user thread of the process stops.
42. What is the primary purpose of thread affinity in thread scheduling? a) To bind a thread to a specific processor b) To allow a thread to change priority c) To enable a thread to move across processors d) To lower the overhead of thread creation	a) To bind a thread to a specific processor Keeps the thread on one CPU so its cache contents remain warm (soft/hard affinity).
43. In thread scheduling, what is the main difference between user-level and kernel-level threads? a) User-level threads are managed by the OS b) Kernel-level threads have higher priority c) User-level threads are managed without OS intervention d) Kernel-level threads are slower to create and manage	c) User-level threads are managed without OS intervention User threads are scheduled by a thread library in user space; kernel threads by the OS scheduler.
44. The model in which one user-level thread is mapped to many kernel-level threads is called _____. a) Many-to-One model b) One-to-Many model c) Many-to-Many model d) One-to-One model	b) One-to-Many model Source key. Note: this is not one of the standard models (Many-to-One, One-to-One, Many-to-Many, Two-Level); the answer follows the wording of the question only.